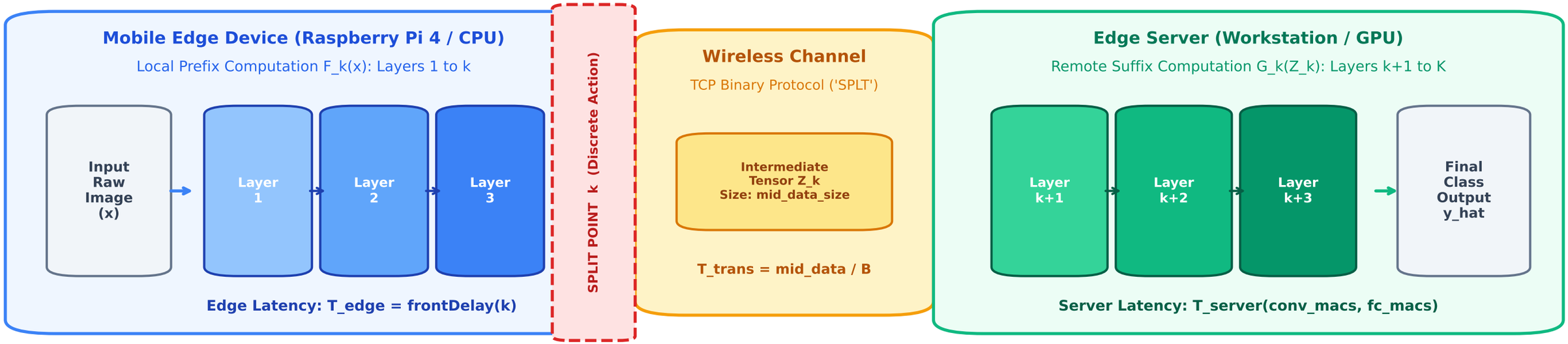
muLinUCB Controller: Sequential Layer Partitioning & Contextual Bandit

Autodidactic Neurosurgeon (ANS) • Online Learning for Deep Inference Offloading

(A) What It Does: Sequential Layer-Wise Partitioning of Deep Neural Networks (CNNs)



Total Sequential Latency: T_total(k) = T_edge(1..k) + T_trans(Z_k) + T_server(k+1..K)

Discrete Split Points: k=0 (Full Local Edge) | k=1..3 (Collaborative Split) | k=4 (Full Remote Server)

(B) How It Works: muLinUCB Contextual Bandit Decision & Online Learning Loop

1. Context Vector

fillThetaContext() in R^7

x[0]: conv_macs (Remote)

x[1]: conv_layers (Count)

x[2]: fc_macs (Remote)

x[3]: fc_layers (Count)

x[4]: act_macs (Remote)

x[5]: act_layers (Count)

x[6]: mid_data_size (Bytes)

2. Ridge Regression & LCB Estimator

Online Matrix State (beta = max(1, Cx^2)):

A = sum(x x^T) + beta * I in R^(7x7)

b = sum(x * y) in R^7

theta_hat = A^(-1) b (Tracks 1/V_server & 1/BW)

Optimistic Delay Prediction L_hat(a):

L_hat(a) = x_a^T theta_hat - alpha * sqrt((1-L) x_a^T A^(-1) x_a)

+ frontDelay(a)

Mean Delay Exploitation | Variance Bonus Exploration

3. Decision Selection

Optimal Split Point k*:

k* = argmin_a L_hat(a)

Forced Sampling:

If t mod forceSample == 0:
argmin over non-local
split arms [0:-1]

Confidence Weighting:
Key Frame L_key = 0.8
Non-Key L_nonkey = 0.2

4. Online Update

Measure Real Latency y_t

y_t = t_loc + t_comm_server

Recursive Updates (updateA_b):

A <- A + x x^T

b <- b + x * y_t

Doubling Trick:
If current_frame > frameNum:
frameNum *= 2, updates alpha

Closed-Loop Feedback: theta_hat tracks network & server load online without external probes